

RAVEL Speed Scaling Evidence

Speed scaling

We report synthetic-token train-step timings to isolate the runtime behavior of the sequence operators from corpus effects. The benchmark uses the same byte-level configuration as the small TinyStories comparison and matches the attention baseline by trainable parameter count. Warmup/compile steps are excluded from the medians. The context-length sweep keeps approximately 1024 tokens per step while increasing T , so the main changing variable is the amount of history visible to each token.

The crossover is sharp. At $T = 128$, compiled RAVEL and attention are close ($1.13\times$ RAVEL/attention throughput). At $T = 512$, RAVEL reaches 132k tokens/s versus 62k for attention ($2.13\times$). At $T = 1024$, the gap grows to $3.94\times$. This is the regime in which the event-memory formulation begins to express its intended scaling advantage: RAVEL’s measured step time remains nearly flat while the attention baseline pays for a growing causal score/value computation.

The fixed-context sweep checks that the effect is not only a single-batch artifact. At $T = 512$, increasing the batch from 1 to 4 raises the compiled RAVEL advantage from $1.74\times$ to $2.62\times$. Thus the speedup persists when the number of tokens per step increases, not merely when the context sweep reduces batch size.

There is one non-monotone point worth keeping rather than smoothing away: at $T = 256$, compiled attention is faster in this CPU/PyTorch setup. This reflects implementation constants and compiler choices, not the asymptotic behavior. The result should therefore be stated as a crossover claim: RAVEL is not universally faster at tiny contexts, but it becomes substantially faster once the context is long enough for attention’s quadratic term to dominate.

RAVEL speedup appears when sequence length, not token count, is increased

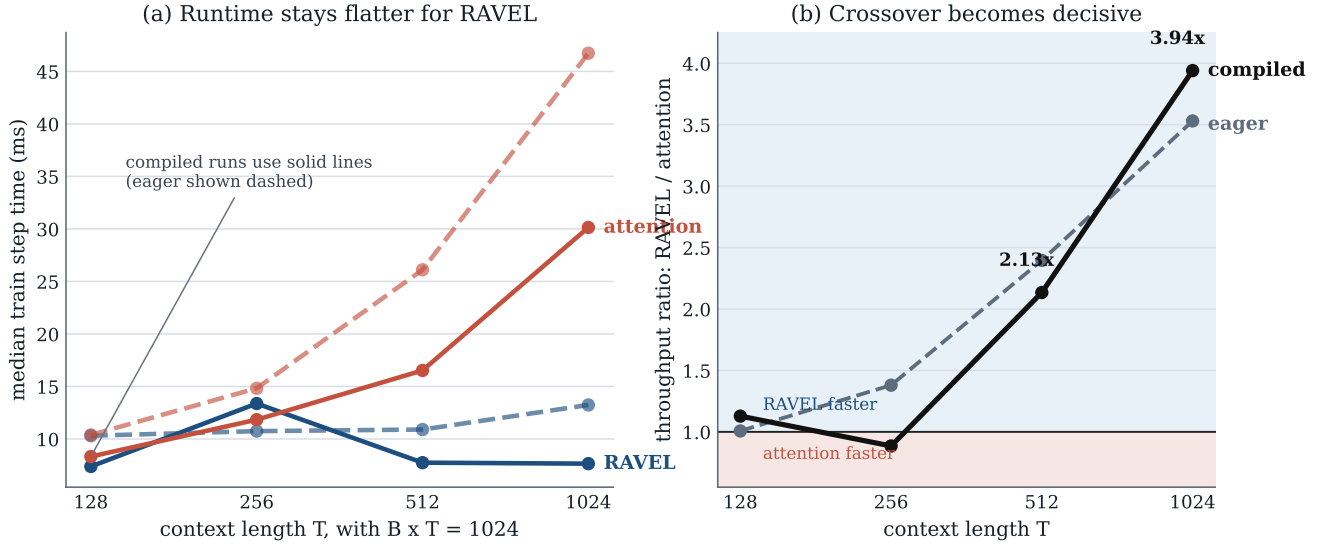


Figure 1: Runtime crossover under a constant-token context sweep. The benchmark holds $BT \approx 1024$ and increases T . RAVEL is near parity at short contexts, but reaches 2.13 $\times$ attention throughput at $T = 512$ and 3.94 $\times$ at $T = 1024$ in compiled mode.

At $T = 512$, RAVEL gains speed as more tokens are packed into each step

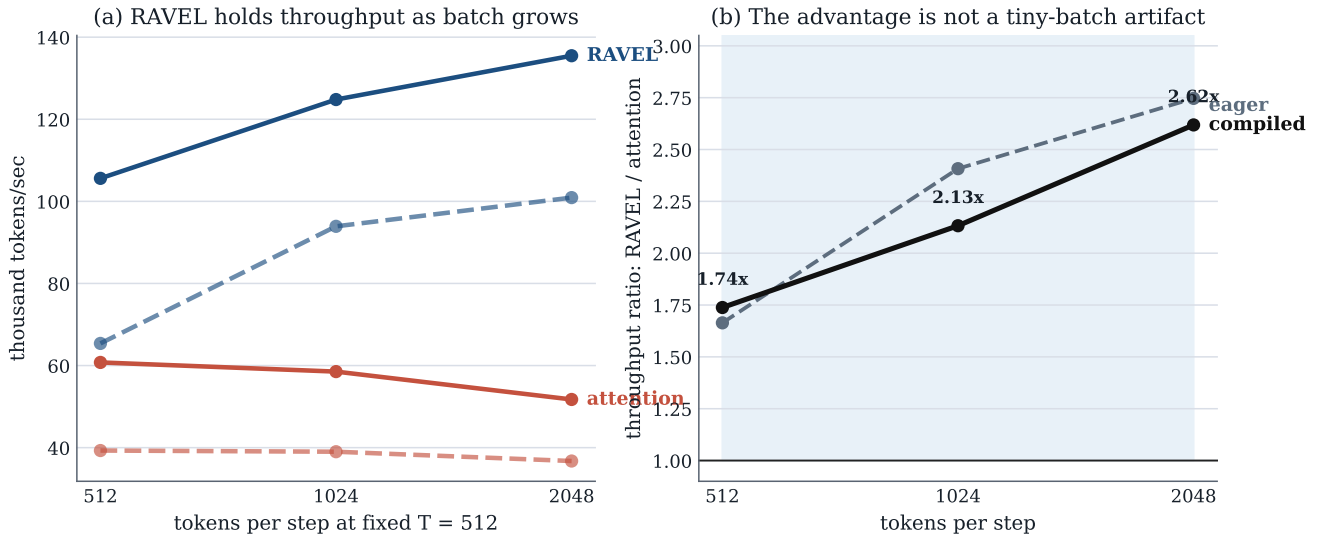


Figure 2: Fixed-context throughput at $T = 512$. As more tokens are packed into each training step, RAVEL's advantage grows rather than disappearing, reaching 2.62 $\times$ attention throughput at 2048 tokens/step in compiled mode.

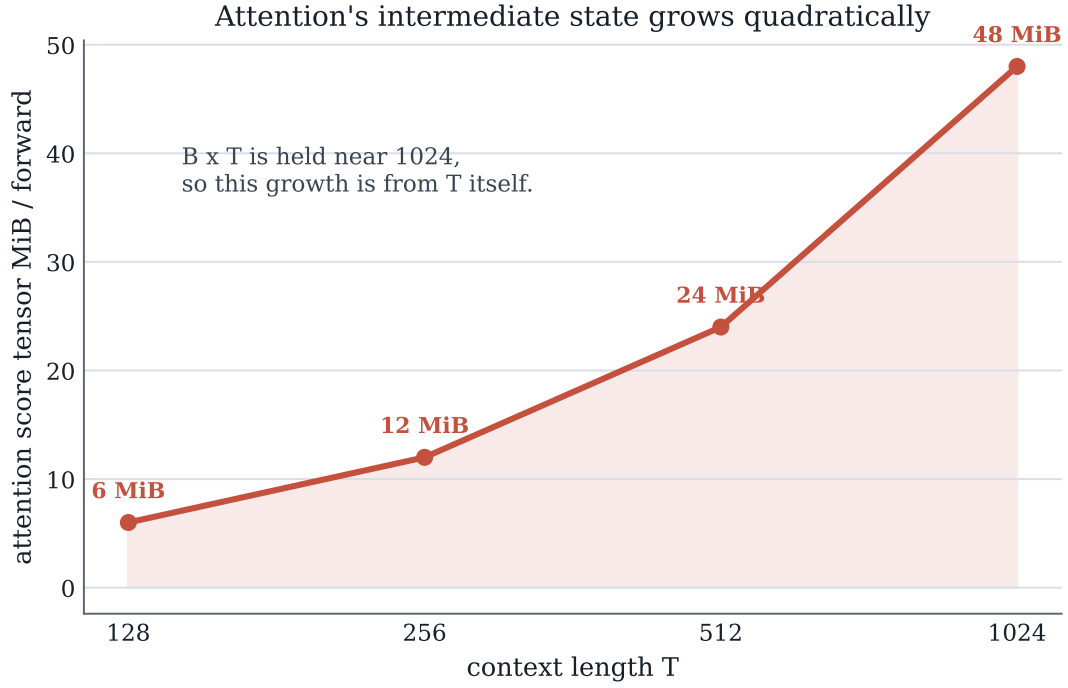


Figure 3: Attention’s causal score tensor grows quadratically with context length. In the context sweep, BT is held near 1024, so the increase comes from T itself rather than from simply processing more tokens per step.

Table 1: Measured RAVEL throughput relative to the matched softmax-attention baseline. Values above 1 indicate RAVEL is faster.

Sweep	Mode	B	T	Tokens/step	RAVEL speedup
context	compiled	8	128	1024	$1.13\times$
context	compiled	4	256	1024	$0.89\times$
context	compiled	2	512	1024	$2.13\times$
context	compiled	1	1024	1024	$3.94\times$
context	eager	8	128	1024	$1.01\times$
context	eager	4	256	1024	$1.38\times$
context	eager	2	512	1024	$2.40\times$
context	eager	1	1024	1024	$3.53\times$
tokens	compiled	1	512	512	$1.74\times$
tokens	compiled	2	512	1024	$2.13\times$
tokens	compiled	4	512	2048	$2.62\times$
tokens	eager	1	512	512	$1.66\times$
tokens	eager	2	512	1024	$2.41\times$
tokens	eager	4	512	2048	$2.75\times$